Neuromorphic Computing with Spiking Neural Networks: Efficient Deployment on Loihi 2 and TrueNorth

SEMINAR REPORT

Submitted by

Abhinav K R

SNM23CA002

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology

in

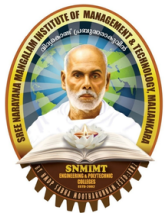
Computer Science and Engineering (Artificial Intelligence)

Under the guidance of

Ms. Anu K A

Assistant Professor

Dept. of CSE



Department of Computer Science and Engineering (Artificial Intelligence)

SREE NARAYANA MANGALAM INSTITUTE OF MANAGEMENT AND TECHNOLOGY

MALIANKARA

October 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE)

SREE NARAYANA MANGALAM INSTITUTE OF MANAGEMENT & TECHNOLOGY, MALIANKARA



CERTIFICATE

This is to certify that the seminar report entitled **Neuromorphic Computing with Spiking Neural Networks: Efficient Deployment on Loihi 2 and TrueNorth** *submitted by* **Abhinav K R** *(***SNM23CA002***), to the* **Department of Computer Science and Engineering (Artificial Intelligence), SNM Institute of Management and Technology, Maliankara**, *in partial fulfilment of the requirement for the award of B.Tech Degree in Computer Science and Engineering (Artificial Intelligence) is a bonafide record of the work carried out by the student.*

Guide

Ms. Anu K A
Asst. Professor
Dept. of CSE

Head of the Department

Ms. Lakshmi Anand
Asst. Professor
Dept. of CSE

Coordinator

Ms. Surya T S
Asst. Professor
Dept. of CSE

Declaration

I undersigned hereby declare that the seminar report **Neuromorphic Computing with Spiking Neural Networks: Efficient Deployment on Loihi 2 and TrueNorth**, submitted for partial fulfilment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Ms. Anu K A**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Maliankara

Date: 22/10/2026

Abhinav K R

SNM23CA002

Acknowledgement

Firstly, I thank God, the Almighty, with whose blessings, this seminar has been successfully completed.

I wish to give deep sense of acknowledgement to our Principal **Dr. Atmaram P**, for his constant encouragement and valuable advice throughout the course.

I am extremely grateful to **Ms. Lakshmi Anand**, Head of the Department of Computer Science and Engineering, for the valuable guidance through this humble endeavor. The highly enterprising attitude, patience for listening to my doubts, timely suggestions and guidance made my seminar a so-called reality in stipulated time. I would also like to thank our seminar coordinator **Ms. Surya T S**, Assistant Professor, Department of Computer Science and Engineering and my seminar guide **Ms. Anu K A**, Assistant Professor, Department of Computer Science and Engineering for their gracious of encouragement and the valuable assistance.

I express my sincere gratitude to all the faculty members of the department of Computer Science and Engineering (Artificial Intelligence) for their co-operation and support to the seminar.

I also express my sincere gratitude to all my friends and my parents for their co-operation and constant inspiration.

Abhinav K R

Abstract

Deep learning has achieved remarkable accuracy, but running it on conventional CPUs and GPUs consumes far more energy than real-time, battery-powered edge systems such as voice interfaces, autonomous vehicles, and surveillance drones can afford. Neuromorphic computing addresses this problem by imitating the brain: Spiking Neural Networks (SNNs) communicate through sparse, discrete events, and neuromorphic processors such as IBM's TrueNorth and Intel's Loihi 2 keep memory next to computation and perform work only when spikes arrive. This seminar presents a comparative study of four recent works that train, convert, and deploy SNNs on these two chips.

Meszaros et al., the base paper, build a complete pipeline that trains SNNs with learnable synaptic delays using the event-based EventProp algorithm in mlGeNN and deploys them on Loihi 2 through the NetX format, improving feedforward keyword-spotting accuracy by up to 26.9 percentage points and running up to 18 times faster and with about 250 times less energy than an NVIDIA Jetson Orin Nano. Isik et al. implement five dataset-specific SNNs for multimodal sensor fusion in autonomous driving on Loihi-2, reporting a power efficiency of 103.94 GOP/s/W and sub-millisecond inference. Shrestha et al. map a Long Short-Term Memory network onto TrueNorth using discretised weights, piecewise-linear gates, burst-coded cell states, and a store-and-release synchronisation mechanism, consuming only 56 μJ per sample. Brehove et al. convert standard quantised PyTorch networks into sigma-delta neural networks that exploit Loihi 2's graded spikes, achieving 16 times lower energy and about 40 times lower energy-delay product than an edge GPU and a real-time field deployment for drone detection.

Comparing these approaches across training strategy, neural coding, hardware constraints, and measured efficiency reveals a consistent pattern: neuromorphic hardware delivers order-of-magnitude gains in energy and energy-delay product when the network's activity is sparse and its computation is mapped to the chip's native primitives—delays, graded spikes, or crossbar cores—while accuracy, latency, and input-output overheads remain the main trade-offs. The study concludes that pairing conventional deep-learning toolchains with hardware-aware conversion and event-based sensors is the most practical path to efficient neuromorphic deployment at the edge.

Keywords: Neuromorphic Computing; Spiking Neural Networks; Intel Loihi 2; IBM TrueNorth; Synaptic Delays; EventProp; Sensor Fusion; Spike-Based LSTM; Sigma-Delta Neural Networks; ANN-to-SNN Conversion; Energy-Efficient Edge AI.

Contents

Acknowledgement **I**

Abstract **II**

List of Figures **VII**

List of Tables **VIII**

List of Abbreviations **IX**

1 Introduction **1**
1.1 General Background 2
1.2 Motivation 3
1.3 Problem Statement 3
1.4 Objectives 3
1.5 Scope 4
1.6 Advantages 4
1.7 Organization of the Report 4

2 Theoretical Foundations **5**
2.1 Neuromorphic Computing Principles 5
2.2 Spiking Neuron Models 6
2.3 Neural Coding Schemes 7
2.4 Training Spiking Neural Networks 8
2.5 Synaptic Delays 8
2.6 Neuromorphic Hardware Platforms 9
2.7 Software Frameworks 10
2.8 Evaluation Metrics 11
2.9 Taxonomy of Approaches 11
2.10 Chapter Summary 12

3 Literature Survey **13**
3.1 A Complete Pipeline for Deploying SNNs with Synaptic Delays on Loihi 2 13
3.1.1 Problem Statement and Motivation 13
3.1.2 Pipeline Architecture 14
3.1.3 Network Models and Datasets 14
3.1.4 Results and Analysis 15
3.1.5 Discussion 17

3.2 Accelerating Sensor Fusion in Neuromorphic Computing: A Case Study on Loihi-2 . . . 17
3.2.1 Problem Statement and Motivation . . . 17
3.2.2 System Architecture . . . 18
3.2.3 Datasets . . . 18
3.2.4 Implementation Framework . . . 19
3.2.5 Results and Analysis . . . 19
3.2.6 Discussion . . . 21
3.3 A Spike-Based Long Short-Term Memory on a Neurosynaptic Processor . . . 21
3.3.1 Problem Statement and Motivation . . . 22
3.3.2 LSTM and the TrueNorth Architecture . . . 22
3.3.3 Constrain-Then-Train-Then-Approximate . . . 22
3.3.4 Spike Encoding . . . 23
3.3.5 Store-and-Release Synchronisation . . . 24
3.3.6 Constituent Modules and Mapping . . . 25
3.3.7 Experiments and Results . . . 25
3.3.8 Discussion . . . 27
3.4 Sigma-Delta Neural Network Conversion on Loihi 2 . . . 27
3.4.1 Problem Statement and Motivation . . . 28
3.4.2 Sigma-Delta Neurons and the Conversion Method . . . 28
3.4.3 Datasets, Models, and Training . . . 30
3.4.4 Results and Analysis . . . 31
3.4.5 Discussion . . . 32
3.5 Summary of the Reviewed Papers . . . 33

4 Discussion & Comparison **34**
4.1 Analytical Focus and Methodology . . . 34
4.2 Training and Conversion Strategies . . . 34
4.3 Neural Coding and Sparsity . . . 35
4.4 Hardware Platforms and Mapping Constraints . . . 35
4.5 Energy, Latency, and Accuracy Trade-offs . . . 36
4.6 Benchmarking Practices and Reproducibility . . . 36
4.7 Overall Comparative Insights . . . 37

5 Applications **38**
5.1 Keyword Spotting and Voice Interfaces . . . 38
5.2 Autonomous Driving and Sensor Fusion . . . 38
5.3 Time-Series and Sequence Processing . . . 38
5.4 Drone Detection and Surveillance . . . 38
5.5 Robotics and Edge IoT . . . 38

6 Challenges and Future Directions **39**
6.1 Challenges Identified in the Reviewed Works 39
6.2 Cross-Cutting Challenges 39
6.3 Future Research Directions 39

7 Conclusion **40**

References **41**

List of Figures

1.1.1 Conventional von Neumann computing compared with event-driven neuromorphic computing 2

2.2.1 Simulated leaky integrate-and-fire neuron: a weak input (green) stays below threshold and leaks away, while a stronger input (orange) produces regular output spikes 6

2.3.1 Neural coding schemes used in the reviewed works: rate and burst codes on TrueNorth [3], spike timing exploited by delays [1], and graded sigma-delta spikes on Loihi 2 [4]; in (d) the grey line is the activation and bar heights are spike payloads 7

2.5.1 Coincidence detection with synaptic delays: inputs that fire at $t_1 < t_2 < t_3$ arrive together because the earliest input has the longest delay 9

2.6.1 Simplified view of a Loihi 2 chip, based on the descriptions in [1, 4] . . . 10

2.9.1 Taxonomy of the neuromorphic SNN deployment approaches reviewed in this seminar 12

3.1.1 The deployment pipeline: models are trained with GeNN and exported in the NetX format for inference on Loihi 2 [1] 14

3.2.1 System integrating the Loihi-2 chip for sensor fusion, and the processing of camera, LIDAR, radar, GPS, and IMU data for autonomous driving [2] 18

3.2.2 Throughput and power efficiency of the CPU, GPU, and Loihi 2 on the sensor-fusion model: the GPU is fastest, but Loihi 2 is far more efficient (data from [2]) 20

3.3.1 (a) Structural view, (b) crossbar view, and (c) corelet abstraction of a TrueNorth core [3] 23

3.3.2 Operation of the spike-based LSTM: input (blue), processing (red), and output (green) partitions alternate between phases, synchronised by store-and-release neurons driven by store, pre-charge, and release clocks [3] 24

3.3.3 Accuracy of the spike-based LSTM on TrueNorth versus phase length; RCt denotes a rate-coded cell state and NoC a network trained without weight constraints [3] 26

3.4.1 (A) A conventional ANN neuron; (B) a quantised sigma-delta neuron on Loihi 2, which accumulates weighted input changes, applies ReLU and integer rescaling, and emits a graded spike Δy when its output has changed by more than a threshold; (C) the workflow from trained ANN to quantised ANN to converted SDNN, with frames turned into delta spikes and output spikes summed into bounding boxes [4] 28
3.4.2 Field deployment at Silent Swarm 2025: (A) infrared frame with a drone and an airplane, (B) a training frame from MSDD, (C) the field setup with an IR camera and the Loihi 2 VPX board, and (D) the system diagram [4] 33

4.5.1 Efficiency advantages over conventional hardware reported in the reviewed works, computed from their published tables: energy or EDP per inference for [1, 3, 4] (feedforward delay network at batch size 1 for [1]) and power efficiency in GOP/s/W for [2] 37

List of Tables

2.1 Comparison of the neuromorphic processors used in the reviewed works . . 10
2.2 Evaluation metrics used in the reviewed works 11

3.1 Test accuracy (%) of models trained on SHD and SSC [1] 16
3.2 SHD inference cost per sample on Loihi 2 and Jetson Orin Nano [1] 17
3.3 Hardware specifications and results on the Oxford Radar RobotCar model [2] 20
3.4 Performance of Loihi-2 on the five sensor-fusion datasets [2] 21
3.5 Power and performance of a 50-unit LSTM on different platforms [3] 27
3.6 Models converted and deployed on Loihi 2 [4] 30
3.7 Loihi 2 compared with Jetson boards at zero threshold [4] 31
3.8 Summary of the reviewed works 33

4.1 Methodological comparison of the reviewed works 35

List of Abbreviations

AER	Address-Event Representation
AI	Artificial Intelligence
AMP	Accumulated Membrane Potential
ANN	Artificial Neural Network
ASIC	Application-Specific Integrated Circuit
BPTT	Backpropagation Through Time
CAN	Controller Area Network
CPE	Corelet Programming Environment
CPU	Central Processing Unit
DL	Deep Learning
EDP	Energy-Delay Product
ERG	Embedded Reber Grammar
FP32	32-bit Floating Point
FPGA	Field-Programmable Gate Array
FPS	Frames Per Second
GOP/s	Giga-Operations per Second
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HDF5	Hierarchical Data Format version 5
IC	Input Collection (module)
IMU	Inertial Measurement Unit
INT8	8-bit Integer
IO	Input/Output
IoT	Internet of Things
LIDAR	Light Detection and Ranging
LIF	Leaky Integrate-and-Fire
LSTM	Long Short-Term Memory
MAC	Multiply-Accumulate
mAP	Mean Average Precision
MSDD	Multi-Sensor Drone Detection (dataset)
MSE	Mean Squared Error

NetX	Network Exchange (format)
PL	Phase Length
RNN	Recurrent Neural Network
RPValue	Represented Value
S-LSTM	Spike-Based Long Short-Term Memory
SDNN	Sigma-Delta Neural Network
SHD	Spiking Heidelberg Digits
SLAYER	Spike Layer Error Reassignment
SNN	Spiking Neural Network
SoC	System on Chip
SRAM	Static Random-Access Memory
SSC	Spiking Speech Commands
SynOps	Synaptic Operations
YOLO	You Only Look Once

Chapter 1

Introduction

Deep neural networks now recognise speech, detect objects, and steer vehicles with remarkable accuracy, but they do so at a high energy cost. A conventional processor stores weights and activations in memory that is physically separate from its arithmetic units, so every layer of a network moves large amounts of data back and forth and performs dense multiply-accumulate (MAC) operations on every input, whether or not that input has changed. On a data-centre GPU this cost is tolerable; on a smart speaker, a car, or a drone running on a battery it is not [2, 4]. The human brain, by contrast, performs perception and control on about twenty watts, because its neurons communicate only through brief, sparse electrical pulses and store their state where they compute.

Neuromorphic computing transfers these principles to silicon. **Spiking Neural Networks** (SNNs)—often called the third generation of neural network models—replace continuous activations with discrete events called *spikes*, and neuromorphic processors execute them on many small cores that each hold the state of their own neurons and synapses. Computation is *event-driven*: a neuron does work only when a spike arrives, and a core sends traffic only when one of its neurons fires. IBM's **TrueNorth** [5] and Intel's **Loihi** [6] and **Loihi 2** [7] are the best-known examples, and the works reviewed here report gains of one to three orders of magnitude in energy over CPUs and GPUs on suitable tasks [1, 2, 3, 4].

This seminar presents a comparative study of four works that deploy SNNs on these chips. Paper [1], the base paper, by Meszaros et al., builds a **complete pipeline** that trains SNNs with learnable **synaptic delays** on GPUs and deploys them on Loihi 2 for keyword spotting. Paper [2], by Isik et al., implements five SNNs for **multimodal sensor fusion** in autonomous driving on Loihi-2 and compares them with a CPU and a GPU.

Paper [3], by Shrestha et al., maps a **Long Short-Term Memory** (LSTM) network onto TrueNorth's spiking cores. Paper [4], by Brehove et al., **converts** conventional quantised networks into **sigma-delta neural networks** that use Loihi 2's graded spikes, and deploys one in the field.

1.1 General Background

Figure 1.1.1 contrasts the two styles of computing. In the **von Neumann** architecture, a processor fetches weights and activations from shared memory over a bus, computes a dense layer, and writes the results back; the time and energy spent moving data—the *memory wall*—dominate the cost of inference. A clock drives every unit at every cycle, and every input frame is processed completely even if it is almost identical to the previous one [4]. In a **neuromorphic** architecture, memory is distributed among many cores, each of which stores the synaptic weights and membrane states of its own neurons in local SRAM. Neurons integrate incoming spikes and fire only when their potential crosses a threshold, and spikes are routed between cores as small packets by a network-on-chip using **address-event representation** (AER) [3]. Energy is therefore spent roughly in proportion to spiking activity rather than to network size.

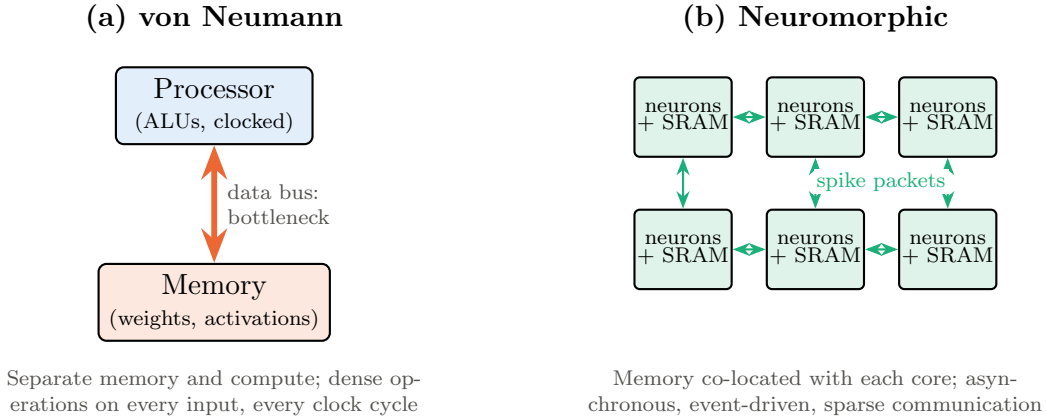


Figure 1.1.1: Conventional von Neumann computing compared with event-driven neuromorphic computing

Two kinds of **sparsity** make this efficient. *Spatial sparsity* means that at any moment only a small fraction of neurons are active, so most synapses do no work. *Temporal sparsity* means that consecutive inputs—audio frames, video frames, sensor readings—change

little, so a network that transmits only changes performs far fewer operations than one that recomputes everything [4]. Exploiting both requires suitable neuron models, suitable ways of encoding numbers in spikes, and training methods that respect the hardware's limited precision, connectivity, and memory. The four reviewed works approach these requirements from different directions: by learning richer *temporal* computation through delays [1], by applying SNNs to a demanding *system-level* task [2], by implementing a complex *recurrent* architecture under severe hardware constraints [3], and by *converting* conventionally trained networks so that they exploit sparsity automatically [4].

1.2 Motivation

Three needs motivate this study. First, **edge intelligence**: keyword spotting, sensor fusion, and drone detection must run continuously on devices with tight power budgets, where cloud processing adds latency, depends on connectivity, and raises privacy concerns [1, 2]. Second, **latency**: autonomous driving and counter-drone systems need decisions within milliseconds, and batching inputs to make a GPU efficient conflicts with this requirement [1, 4]. Third, **programmability**: neuromorphic chips have been available to researchers for years, yet their adoption is limited because training SNNs is harder than training conventional networks and because each chip imposes its own constraints on weights, neurons, and connectivity [3, 4]. A clear picture of how recent works overcome these obstacles is therefore valuable.

1.3 Problem Statement

Given a task that is naturally temporal or streaming—speech, multimodal sensor data, symbol sequences, or video—obtain a spiking neural network that achieves accuracy comparable to a conventional deep network, map it onto a neuromorphic processor within that processor's constraints on precision, connectivity, and neuron resources, and demonstrate lower energy, latency, or energy-delay product than conventional hardware.

1.4 Objectives

The objectives of this seminar are:

- To introduce spiking neuron models, neural coding schemes, SNN training and con-

version methods, and the TrueNorth and Loihi 2 architectures.

- To review in depth four works on SNN deployment for keyword spotting, sensor fusion, sequence learning, and video understanding.
- To compare their training strategies, coding schemes, hardware mappings, and measured efficiency.
- To identify open challenges and future research directions for practical neuromorphic edge computing.

1.5 Scope

The seminar covers delay learning and deployment on Loihi 2 [1], sensor fusion for autonomous driving on Loihi-2 [2], spike-based recurrent networks on TrueNorth [3], and ANN-to-SNN conversion with graded spikes on Loihi 2 [4], together with the datasets, software frameworks, and metrics that these works use. Analogue and mixed-signal neuromorphic chips and on-chip learning are discussed only briefly.

1.6 Advantages

Neuromorphic deployment of SNNs offers several advantages over conventional deep-learning hardware:

- **Energy Efficiency:** work is done only when spikes occur, so sparse workloads consume a small fraction of GPU energy.
- **Low Latency:** inputs are processed as they arrive, without waiting to fill a batch.
- **Native Temporal Processing:** membrane dynamics and synaptic delays give networks built-in memory for time-varying signals.
- **Scalability:** chips tile into boards of many cores and chips that run larger networks without a shared memory bottleneck.

1.7 Organization of the Report

Chapter 2 introduces the theoretical foundations. **Chapter 3** reviews the four works in detail. **Chapter 4** compares the approaches, **Chapter 5** discusses applications, **Chapter 6** presents challenges and future directions, and **Chapter 7** concludes the report.

Chapter 2

Theoretical Foundations

This chapter introduces the concepts on which the reviewed works build: the principles of neuromorphic computing, spiking neuron models, ways of encoding numbers in spikes, methods for training and converting SNNs, synaptic delays, the TrueNorth and Loihi 2 processors and their software, and the metrics used to evaluate neuromorphic systems.

2.1 Neuromorphic Computing Principles

Neuromorphic systems are built on three principles. The first is **event-driven computation**: instead of a global clock forcing every unit to compute at every cycle, a neuron updates its state when it receives a spike, and a spike is sent only when a neuron fires [3]. The second is **co-location of memory and compute**: each core keeps its neurons' weights and internal states in local SRAM, which avoids the costly transfers between processor and external memory that dominate conventional inference [4]. The third is **massive parallelism**: thousands of simple cores operate concurrently and exchange spikes asynchronously through a network-on-chip, so that larger networks are handled by adding cores or chips rather than by enlarging a single processor [2].

These principles make the energy of a neuromorphic computation roughly proportional to the number of **synaptic operations** (SynOps)—the number of spikes multiplied by their fan-out—rather than to the number of MACs in the equivalent dense network. The efficiency gain therefore depends on how sparse the network's activity is: a network that fires densely gains little, whereas one that fires rarely, or only when its input changes, can save one or more orders of magnitude [4]. Much of the work reviewed in Chapter 3 is about obtaining sparse activity without losing accuracy.

2.2 Spiking Neuron Models

The most widely used neuron model in neuromorphic hardware is the **Leaky Integrate-and-Fire** (LIF) neuron, used directly in [1] and in variant forms on both chips. Its membrane potential $V(t)$ integrates an input current $I(t)$ while leaking towards a resting value:

$$\tau_m \frac{dV}{dt} = -\big(V - V_{\text{rest}}\big) + R\,I(t), \tag{2.1}$$

where τ_m is the membrane time constant and R the membrane resistance. When V reaches a threshold ϑ, the neuron emits a spike and V is reset to V_{reset}. In discrete time, as executed on chip, the update for neuron j becomes

$$V_j[t] = \alpha\, V_j[t-1] + \sum_i w_{ij}\, s_i[t], \qquad s_j[t] = \begin{cases} 1, & V_j[t] \geq \vartheta \\ 0, & \text{otherwise} \end{cases} \tag{2.2}$$

with decay factor $\alpha = e^{-\Delta t/\tau_m}$, synaptic weights w_{ij}, and binary input spikes $s_i[t] \in \{0, 1\}$. With **exponential synapses**, each input spike instead adds to a synaptic current that itself decays with time constant τ_s, smoothing the input [1]. Figure 2.2.1 shows the resulting behaviour: a weak input charges the membrane without reaching threshold and the potential leaks away when the input stops, whereas a stronger input drives the neuron to fire regularly, with a rate that increases with input strength.

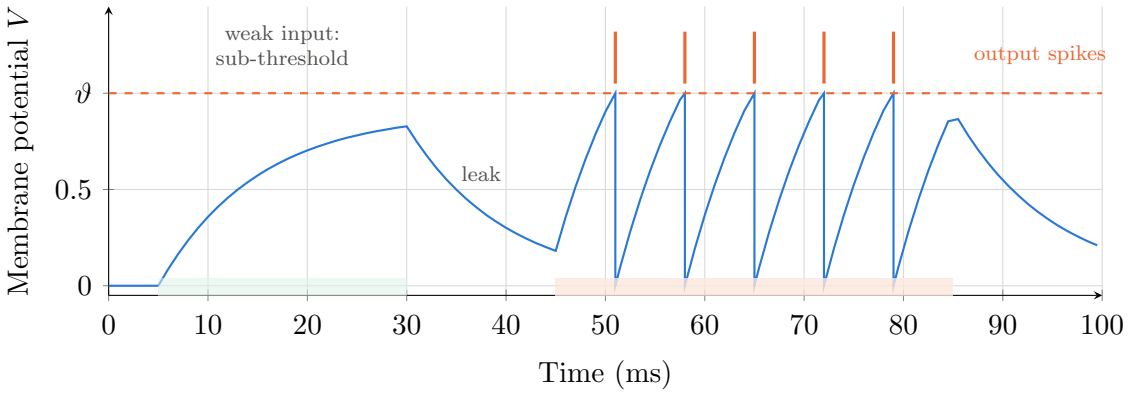


Figure 2.2.1: Simulated leaky integrate-and-fire neuron: a weak input (green) stays below threshold and leaks away, while a stronger input (orange) produces regular output spikes

Other neuron models are also used. TrueNorth neurons are configurable integrate-and-fire units with linear leak, stochastic thresholds, and saturating negative thresholds, which [3] exploits to build store-and-release circuits. Loihi 2 neurons are **programmable**: a small microcode program defines how the neuron updates its state, so new models such as the sigma-delta neuron of [4] can be implemented without changing the hardware. Output layers are often **non-firing leaky integrators** whose potentials are read out directly; in [1] the class decision is the output neuron with the largest exponentially weighted integral of its voltage over the sample.

2.3 Neural Coding Schemes

Because spikes are all-or-nothing events, numerical values must be *encoded* in their timing or number. Figure 2.3.1 illustrates the four schemes that appear in the reviewed works. In **rate coding**, a value is represented by the number of spikes in a window, often generated stochastically; it is robust but needs long windows for high precision [3]. In **burst coding**, the value is the number of spikes emitted in a consecutive burst, which correlates more tightly with the represented value than random rate codes and is used in [3] for the LSTM cell state. In **temporal coding**, information lies in precise spike times, which delays exploit [1]. Finally, Loihi 2 supports **graded spikes** that carry an integer payload; in **sigma-delta coding** a neuron sends a graded spike containing the *change* in its activation only when that change exceeds a threshold, so a static input produces no traffic at all [8, 4].

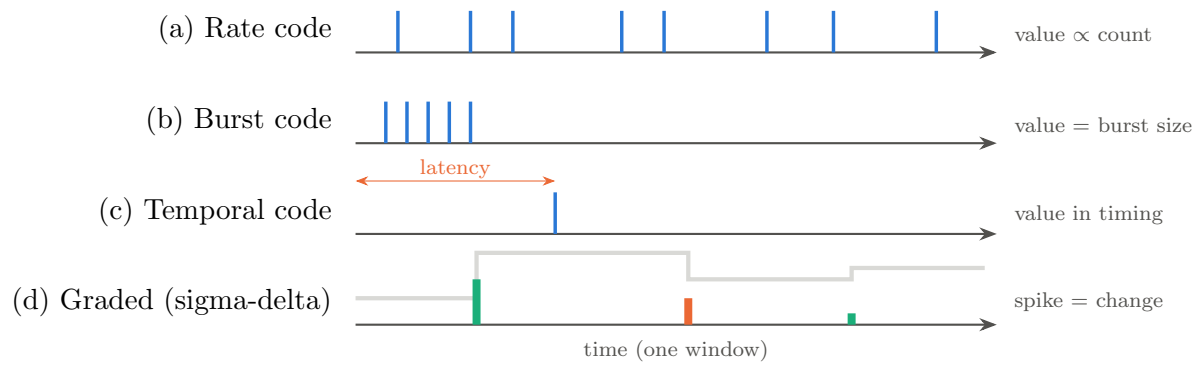


Figure 2.3.1: Neural coding schemes used in the reviewed works: rate and burst codes on TrueNorth [3], spike timing exploited by delays [1], and graded sigma-delta spikes on Loihi 2 [4]; in (d) the grey line is the activation and bar heights are spike payloads

2.4 Training Spiking Neural Networks

The spike function is a step, so its derivative is zero almost everywhere and standard backpropagation cannot be applied directly. Two families of methods address this. **Direct training** treats the SNN as a recurrent network unrolled in time and trains it with **Backpropagation Through Time** (BPTT). *Surrogate-gradient* methods replace the spike derivative with a smooth function; **SLAYER** [9], part of Intel's Lava-DL library, assigns errors back through time in this way and is used to train sigma-delta networks for Loihi 2. These methods must store the whole time series of every neuron, which makes training memory-hungry and slow [4]. **EventProp** [10] instead computes *exact* gradients with an adjoint system that runs backwards in time and needs to store neuron states only at spike times. Because its backward pass has the same event-driven structure as the forward pass, it can be implemented efficiently on GPUs in the mlGeNN library and extended to learn synaptic delays [1].

ANN-to-SNN conversion sidesteps spike gradients: a conventional network is trained with standard tools and then translated into an SNN. Classic conversion represents each ReLU activation by a spike rate, which needs many timesteps per input for accurate rates. Paper [4] instead converts a quantised network into a sigma-delta network whose graded spikes carry exact integer changes, while [3] trains a conventional LSTM under hardware-motivated constraints and then approximates it with spiking circuits.

2.5 Synaptic Delays

In biological networks, spikes take different amounts of time to travel along different axons, and these **synaptic delays** contribute to learning and to detecting coincident inputs [1]. Adding a delay d_{ij} to each connection changes the input to neuron j to

$$I_j(t) = \sum_i w_{ij}\, s_i\big(t - d_{ij}\big), \tag{2.3}$$

so a neuron can respond to a particular *temporal pattern* of input spikes rather than only to their total. Figure 2.5.1 shows the idea: three inputs that fire at different times reach the target simultaneously because each connection has a different delay, and the coincident

arrival drives the neuron above threshold. Networks with learnable delays can compute a richer class of functions than networks with learnable weights alone, and they achieve state-of-the-art results in keyword recognition [1]. Loihi 2 supports synaptic delays of up to 62 timesteps in hardware.

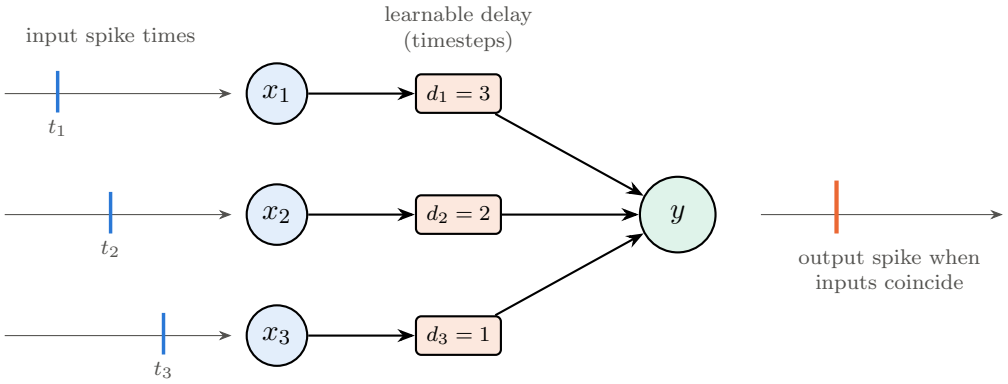


Figure 2.5.1: Coincidence detection with synaptic delays: inputs that fire at $t_1 < t_2 < t_3$ arrive together because the earliest input has the longest delay

2.6 Neuromorphic Hardware Platforms

IBM TrueNorth [5] consists of 4096 neurosynaptic cores, each with 256 input axons, 256 neurons, and a 256×256 crossbar of programmable binary synapses, giving one million neurons and 268 million synapses. Each neuron can use at most four different synaptic weight values, selected by the type of the input axon, and weights are 9-bit signed integers. The chip normally advances in 1 ms *ticks*, within which membrane potentials are updated and spikes are routed asynchronously; a spike can target any single axon on the chip, and active power is proportional to firing activity [3]. Networks are described as **corelets**—reusable blocks of cores with defined inputs and outputs—in the object-oriented Corelet Programming Environment.

Intel Loihi 2 [7], the successor of Loihi [6], is a fully asynchronous chip fabricated on the Intel 4 process. It contains 120 neuromorphic cores, each supporting about 8k neurons whose weights and states are held in local SRAM, together with six embedded processor cores and a 10 Gbps Ethernet spike interface [1, 4]. Its key features for the reviewed works are **programmable microcode neurons**, **graded spikes** with integer payloads of up to 16 bits, and **synaptic delays** of up to 62 timesteps. Chips are deployed on the

single-chip Oheo Gulch board, the 8-chip Kapoho Point board—which, according to [2], provides 1024 cores and 8.4 million neurons—and a 16-chip VPX board [4]. Figure 2.6.1 sketches the chip, and Table 2.1 compares the two processors.

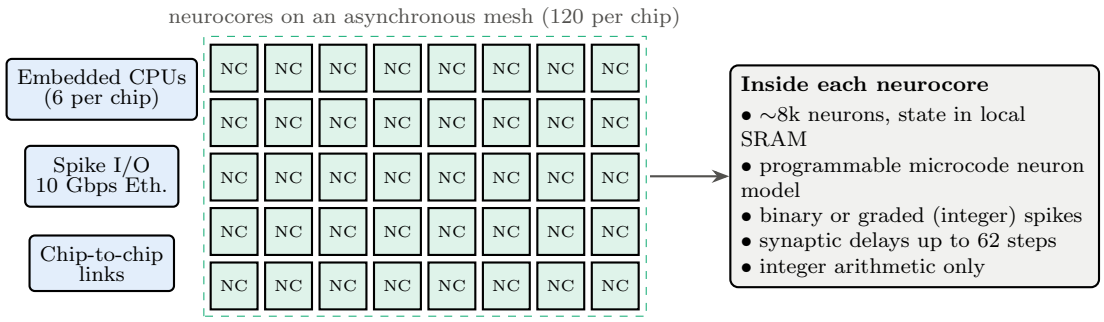


Figure 2.6.1: Simplified view of a Loihi 2 chip, based on the descriptions in [1, 4]

Table 2.1: Comparison of the neuromorphic processors used in the reviewed works

Feature	**IBM TrueNorth [3, 5]**	**Intel Loihi 2 [1, 2, 4]**
Cores per chip	4096	120 (128 reported in [2])
Neurons	256 per core; 1 million per chip	∼8k per core
Synapses	256×256 binary crossbar per core; 268 million	Configurable; weights in local SRAM
Weight precision	9-bit signed; at most 4 values per neuron	Integer; 8-bit weights in [1, 4]
Spikes	Binary	Binary or graded (integer payload)
Neuron model	Fixed, configurable integrate-and-fire	Programmable microcode
Delays	Configurable axonal delays	Up to 62 timesteps
Time base	1 ms ticks (faster than real time possible)	Algorithmic timesteps, asynchronous
Programming	Corelet Programming Environment	Lava, Lava-DL, NetX, NxKernel

2.7 Software Frameworks

Software determines how easily a network reaches the chip. For Loihi 2, Intel's open-source **Lava** framework provides processes and a runtime, and **Lava-DL** adds SLAYER training, a rate-based *Bootstrap* method, and **NetX**, an HDF5-based format for importing trained models [2]. **NxKernel**, available to members of the Intel Neuromorphic Research Community, maps networks to cores efficiently and provides power and timing probes [1, 4]. **mlGeNN**, built on the GPU-optimised GeNN simulator, supports EventProp training

and exports to NetX [1]. TrueNorth networks are composed as corelets in the **Corelet Programming Environment** [3], and [4] shows that standard **PyTorch** training and quantisation can be the front end of a neuromorphic pipeline.

2.8 Evaluation Metrics

Neuromorphic systems are judged on task quality and on efficiency, and the reviewed works use several metrics (Table 2.2). Task quality is measured by **classification accuracy**, **mean average precision** (mAP) for detection, and **mean squared error** (MSE) for regression. Efficiency is measured by **energy per inference**, split into *static* energy, consumed whenever the chip is powered, and *dynamic* energy, caused by computation; by **latency** from input to output; by **throughput** in frames or operations per second; and by **power efficiency** in operations per second per watt. Because designs can trade speed for energy, the **energy-delay product** (EDP),

$$\mathrm{EDP} = E_{\mathrm{inference}} \times T_{\mathrm{latency}}, \tag{2.4}$$

summarises both, with lower values being better [1, 4]. Finally, **sparsity** is measured as the ratio of synaptic operations on the chip to the MACs of the equivalent ANN [4].

Table 2.2: Evaluation metrics used in the reviewed works

Metric	**Meaning**	**Used in**
Accuracy, mAP, MSE	Task quality (classification, detection, regression)	[1, 3, 4]
Energy per inference	Total, static, and dynamic energy per sample or frame	[1, 3, 4]
Latency	Time from input to corresponding output	[1, 2, 3, 4]
Throughput	Frames per second or GOP/s	[2, 4]
Power efficiency	GOP/s/W or pJ per operation	[2]
Energy-delay product	Energy × latency (lower is better)	[1, 4]
Sparsity, resources	SynOps ratio, cores used, neuron density	[3, 4]

2.9 Taxonomy of Approaches

Figure 2.9.1 organises the reviewed works along four dimensions: how the spiking network is obtained, how information is coded, which hardware it targets, and which application it serves. The works are complementary: [1] and [4] represent the two main training

routes—direct event-based training and conversion—while [3] shows how a complex recurrent architecture can be built from spiking circuits, and [2] studies the system-level benefit of neuromorphic hardware for a demanding application.

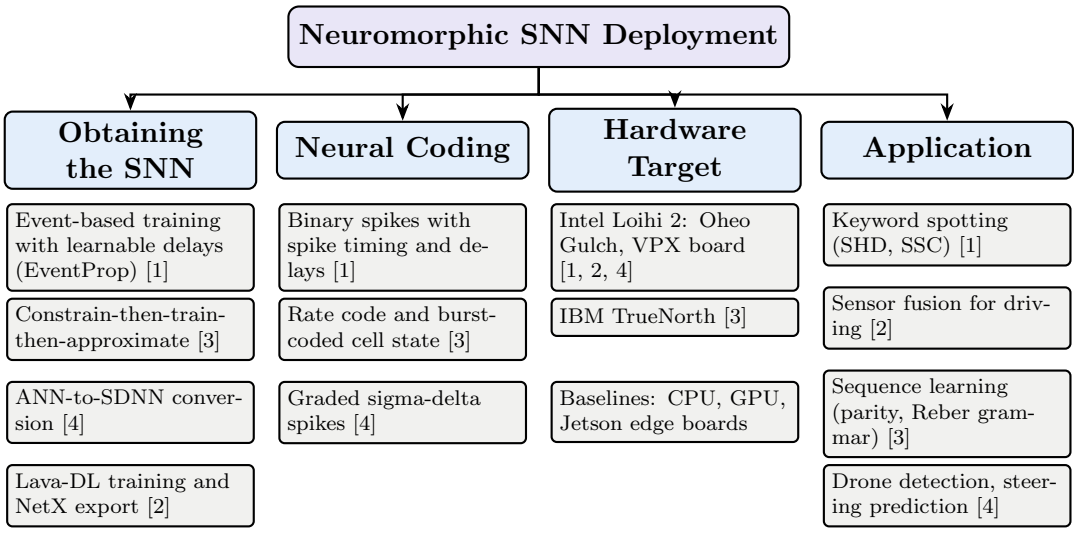


Figure 2.9.1: Taxonomy of the neuromorphic SNN deployment approaches reviewed in this seminar

2.10 Chapter Summary

SNNs compute with sparse events, and neuromorphic processors turn this sparsity into energy savings. Obtaining accurate SNNs requires direct training or conversion, and each choice interacts with the hardware: TrueNorth offers many simple, tightly constrained cores, while Loihi 2 offers programmable neurons, graded spikes, and delays.

Chapter 3

Literature Survey

This chapter reviews the four works in detail. For each, it describes the problem and motivation, the method, the experimental setup, and the results, and closes with a short discussion of strengths and limitations.

3.1 A Complete Pipeline for Deploying SNNs with Synaptic Delays on Loihi 2

The base paper, by Meszaros, Knight, Timcheck, and Nowotny [1], from the University of Sussex and Intel Labs, presents an end-to-end pipeline that trains SNNs with learnable synaptic delays on GPUs and deploys them on Loihi 2 for keyword recognition.

3.1.1 Problem Statement and Motivation

Keyword recognition underlies voice control in smart speakers, phones, and wearables. Sending audio to the cloud adds latency, needs a stable connection, and raises privacy concerns, so efficient on-device processing is desirable [1]. Neuromorphic hardware is well suited to such temporal tasks, but a key limitation has been the lack of support for end-to-end learning that targets the chip. Delays are an attractive ingredient because they let networks compute richer temporal functions, and Loihi implements them efficiently. Existing delay-learning methods, however, either do not exploit the sparsity of SNNs—such as the dilated-convolution method of Hammouamri et al.—or had not been deployed on hardware. The authors' earlier work extended **EventProp** to learn delays with exact gradients in multi-layer and recurrent networks; this paper completes the path to the chip and is, to the authors' knowledge, the first deployment of SNNs with trained delays on neuromorphic hardware for keyword recognition.

3.1.2 Pipeline Architecture

Figure 3.1.1 shows the three stages. Networks are trained on GPUs in **mlGeNN**, built on the GeNN simulator, using EventProp, which stores neuron states only at spike times and has a backward pass with the same event-driven structure as the forward pass. Trained models are exported to **NetX**, the HDF5-based exchange format of the Loihi ecosystem; the authors extended an existing mlGeNN-to-NetX exporter to support delays and to clamp them below 62 timesteps, the hardware limit. Finally, the NetX models are deployed on Loihi 2 through **NxKernel** with weights quantised to 8-bit integers [1]. Algorithm 1 summarises the procedure.

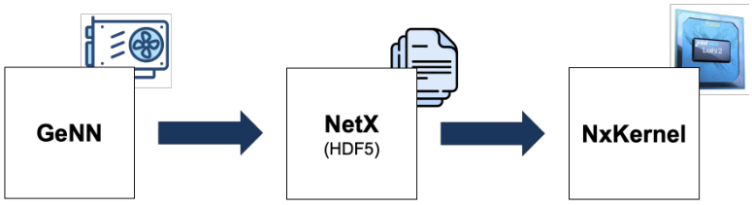


Figure 3.1.1: The deployment pipeline: models are trained with GeNN and exported in the NetX format for inference on Loihi 2 [1]

Algorithm 1 Training and deploying delay SNNs on Loihi 2 (summarised from [1])

Require: spike-encoded dataset (700 input channels); architecture; delay limit $D_{\max} = 62$ timesteps

1: Build an LIF network with exponential synapses, weights w_{ij}, and delays d_{ij} in mlGeNN
2: **for** each training epoch **do**
3: Simulate the forward pass on the GPU, recording neuron states at spike times
4: Run the EventProp adjoint backwards in time to obtain exact gradients for w_{ij} and d_{ij}
5: Update weights and delays; regularise hidden firing towards 14 Hz
6: **end for**
7: Clamp delays to $d_{ij} < D_{\max}$ and export the network to NetX (HDF5)
8: Quantise weights to 8-bit integers and map the network onto Loihi 2 cores with NxKernel
9: For each input, classify by the output neuron with the largest weighted voltage integral

3.1.3 Network Models and Datasets

Two architectures are trained, each with and without delays: a **single-layer recurrent** network and a **two-layer feedforward** network of LIF neurons with exponential

synapses. Output neurons are non-firing leaky integrators, and the decision is the output with the maximum exponentially weighted voltage integral. Hidden layers have 512 neurons for SHD and 1024 for SSC. Hyperparameter searches covered timesteps of 1–2 ms, delay initialisation ranges, and regularisation strengths, with a target hidden firing rate of 14 Hz; cross-validation on the training set was used for SHD, which has no validation set, and the validation set for SSC [1].

Both datasets come from the Heidelberg spiking data sets of Cramer et al. The **Spiking Heidelberg Digits** (SHD) contain 10,420 spoken digits (0–9) in German and English from 12 speakers. The **Spiking Speech Commands** (SSC) contain 105,829 samples of 35 words from Google Speech Commands, spoken by a larger pool of speakers. In both, audio is converted into spikes on 700 input channels by an artificial cochlea model [1].

3.1.4 Results and Analysis

Training efficiency. Compared with the dilated-convolution method on two-hidden-layer feedforward models, mlGeNN's EventProp delay learning needs only half the memory, even with the 62 delay slots used here, and runs up to 10 times faster [1].

Accuracy. Table 3.1 lists test accuracies with 32-bit floating-point weights in mlGeNN (mean of five seeds) and with 8-bit integer weights on Loihi 2 (one seed). The authors report that delays bring only modest gains for recurrent networks—1.0 percentage points on SHD and 1.6 on SSC—but large gains for feedforward networks, 13.3 points on SHD and 26.9 on SSC, so that the feedforward network with delays achieves the highest SSC accuracy [1]. Loihi 2 accuracy closely tracks the GPU results, confirming that quantisation to 8-bit costs little; the single-seed Loihi results are even higher in some cases, such as 90.9% for the recurrent SHD network with delays. Some differences in the GPU columns of the table, such as 72.1% to 86.9% for feedforward SHD, are larger than the gains quoted in the text, so the stated percentages should be read as approximate.

Cost of the delay limit. In the authors' earlier work, models with unconstrained delays outperformed the 62-step models presented here by about 5% on both datasets. At

Table 3.1: Test accuracy (%) of models trained on SHD and SSC [1]

Dataset	Architecture	mlGeNN (GPU, FP32)		Loihi 2 (INT8)	
		No delay	Delay	No delay	Delay
SHD	Feedforward	72.1 ± 2.6	86.9 ± 2.3	67.9	88.0
	Recurrent	87.0 ± 2.1	88.8 ± 2.1	89.5	90.9
SSC	Feedforward	44.5 ± 1.1	71.4 ± 1.0	42.6	69.8
	Recurrent	63.7 ± 1.0	65.3 ± 5.7	62.5	67.8

the 1 ms timestep that gave the best results, 62 steps cover only 62 ms of history, which may be a heavy limitation for keyword recognition [1].

Energy and latency. Table 3.2 compares SHD inference on Loihi 2 with an NVIDIA Jetson Orin Nano. On the Jetson, runtime was measured with CUDA events and power with the `jtop` tool over the CPU, GPU, CV, and SoC rails, with static power taken from 20 s of idling. At batch size 1 the GPU is under-used and latency is dominated by launching three kernels per timestep, about 10 μs each, so energy per sample is roughly constant across architectures. Larger batches amortise this overhead and reduce energy per sample by up to 20 times for models without delays, but delayed inputs must be accumulated in global memory and the delay buffers must fit in the 2 MiB L2 cache, so the best batch size for delay models is small and delays cost about 6 times more energy on the Jetson. On Loihi 2, measured on a single-chip Oheo Gulch system in IO-unconstrained mode, delays only marginally increase energy and latency, and the feedforward networks use about 30% more energy than the recurrent ones because they have twice as many hidden neurons [1].

At batch size 1, Loihi 2 classifies a sample up to 18 times faster than the Jetson—1.46 ms against 27.5 ms for the feedforward delay network—and uses 200 to 260 times less total energy, the roughly 250-fold reduction quoted by the authors. Even against the Jetson's most efficient batched configurations, which raise latency to as much as 179 ms, Loihi 2 uses 20 to 116 times less energy per sample, and its energy-delay product is more than 2000 times lower [1].

Table 3.2: SHD inference cost per sample on Loihi 2 and Jetson Orin Nano [1]

Arch.	Delay	Hardware	Total (mJ)	Dyn. (mJ)	Lat. (ms)	EDP (μJ·s)
FF	Yes	Jetson, batch 1	93.7	30.0	27.5	2581.9
	No	Jetson, batch 1	85.4	25.8	26.0	2219.6
	Yes	Jetson, batch 4	50.6	22.6	48.5	2454.8
	No	Jetson, batch 128	8.3	4.9	178.8	1476.9
	Yes	Loihi 2	0.46	0.28	1.46	0.67
	No	Loihi 2	0.42	0.25	1.47	0.62
Rec	Yes	Jetson, batch 1	93.3	29.0	27.9	2599.0
	No	Jetson, batch 1	85.0	25.3	26.0	2214.8
	Yes	Jetson, batch 4	41.9	16.5	43.6	1827.6
	No	Jetson, batch 128	6.5	3.7	156.4	1018.2
	Yes	Loihi 2	0.36	0.21	1.54	0.56
	No	Loihi 2	0.33	0.19	1.49	0.50

3.1.5 Discussion

The paper's main strength is completeness: it connects a memory-efficient, exact-gradient training method to a real chip and shows that delays, which are expensive on GPUs, are almost free on Loihi 2. It also demonstrates that 8-bit quantisation preserves accuracy. Its limitations are the 62-step delay limit, which costs about 5% accuracy; the restriction to keyword spotting with fairly large, fully connected layers; Loihi results from a single seed; and an IO-unconstrained measurement mode that excludes the cost of streaming inputs to the chip. The authors propose sparsely connected delay networks, combinations of delays with adaptive or resonate-and-fire neurons, and tasks that need precise timing over long periods as future work [1].

3.2 Accelerating Sensor Fusion in Neuromorphic Computing: A Case Study on Loihi-2

Isik, Tiwari, Eryilmaz, and Dikmen [2], from Drexel University, IISc Bangalore, Bilkent University, and the Temsa R&D Center, study whether Loihi-2 can accelerate multimodal sensor fusion for autonomous driving and robotics.

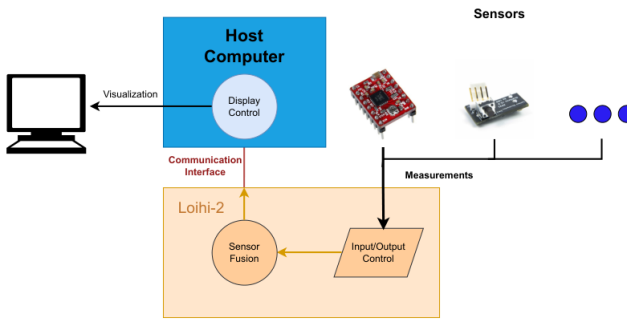
3.2.1 Problem Statement and Motivation

Sensor fusion integrates data from several sensors—cameras, LIDAR, radar, GPS, inertial measurement units (IMUs), and vehicle buses—to obtain a more accurate and

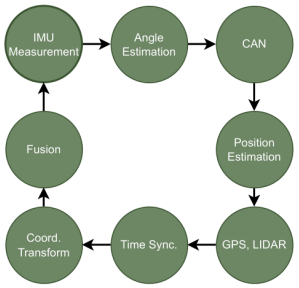
reliable view of the environment than any single sensor provides. Classical approaches include rule-based systems, statistical estimators such as Kalman filters, and neural networks. The task is hard because sensors produce data in different formats and at different rates, calibration discrepancies and noise add uncertainty, data volumes are large, and decisions must be made in real time [2]. Traditional computing is increasingly strained by these demands, and the authors identify a gap in the literature on applying neuromorphic chips such as Loihi-2 to sensor fusion. Their objectives are to demonstrate Loihi-2's efficacy for sensor fusion and to compare it with conventional CPUs and GPUs.

3.2.2 System Architecture

Figure 3.2.1a shows the system. A **host computer** provides visualisation and display control and connects to the chip through a **communication interface**, such as Ethernet. Loihi-2 sits at the centre, performing sensor fusion and managing input and output, while the sensors act as the system's data sources, sending measurements that are likely encoded as spikes [2]. Figure 3.2.1b shows the sensing pipeline for autonomous driving: IMU measurements, angle estimation, CAN-bus data, position estimation, GPS and LIDAR, time synchronisation, and coordinate transformation all feed the fusion stage executed on the neuromorphic unit.



(a) System architecture



(b) Sensing and fusion pipeline

Figure 3.2.1: System integrating the Loihi-2 chip for sensor fusion, and the processing of camera, LIDAR, radar, GPS, and IMU data for autonomous driving [2]

3.2.3 Datasets

Five autonomous-driving datasets, complemented by synthetic data for specific conditions, are used. They differ in sensor modalities and focus, from environmental perception

to driver behaviour, which makes them a varied test of multimodal processing [2].

3.2.4 Implementation Framework

Five SNN models, one tailored to each dataset, were first run on conventional CPU and GPU hardware to validate them and then optimised for Loihi-2. Deployment requires generating an **HDF5** file that captures the trained model's architecture and parameters in a form the Loihi-2 environment accepts, and a script that defines input and output ports and sets up execution on the chip. The authors use **Lava-DL**, including its SLAYER and Bootstrap libraries, together with `lava-dl netx`, `lava.magma.core`, `lava.process`, and `lava.util`. A notable practical difficulty was a mismatch between the data loader's output and the input formats expected by Loihi-2, which the authors illustrate with a model trained on CIFAR-10 [2].

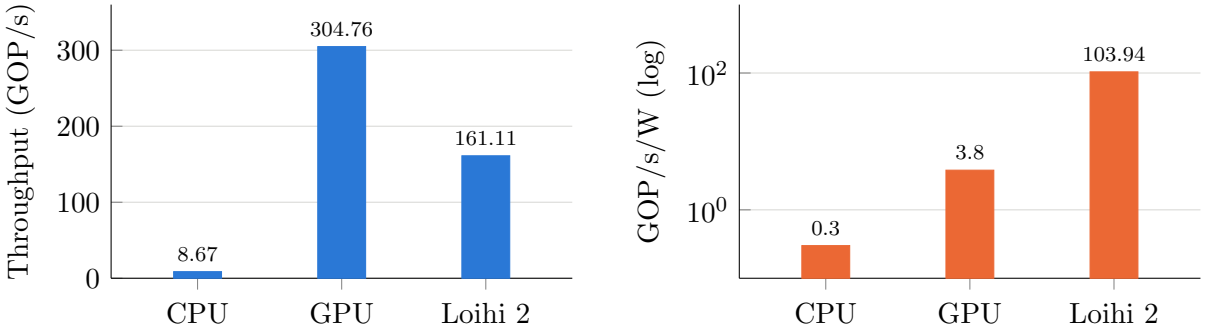
The authors place this flow in a broader architecture in which frameworks such as Lava, Neurokernel, PyNN, Brian2, Nengo, and NEURON translate algorithms for neuromorphic execution, communication uses **NxLink** and **AER**, and sensors, neuromorphic processing units, and actuators form a closed loop. They also list four practical considerations: exploiting **sparsity**, **simplifying neuron models** to fit the chip's dynamics, ensuring **software–hardware compatibility** with Loihi's software stack, and using Intel's **neuromorphic debugging tools** [2].

3.2.5 Results and Analysis

Hardware comparison. Table 3.3 compares an Intel i9-12900H CPU, an NVIDIA RTX 3060 GPU, and Loihi 2 running the model for the Oxford Radar RobotCar dataset, with the same 960 million operations per inference on each. Loihi 2 reaches 161.11 GOP/s at only 1.55 W, a power efficiency of **103.94 GOP/s/W**. The GPU has higher throughput, 304.76 GOP/s, thanks to 3584 cores at 1320 MHz, but draws 80 W, giving 3.80 GOP/s/W, while the CPU achieves 8.67 GOP/s at 28 W, or 0.30 GOP/s/W [2]. Loihi 2 is therefore about 27 times more power-efficient than the GPU and over 300 times more than the CPU (Figure 3.2.2); the authors summarise this as nearly 30 times and over 100 times, respectively.

Table 3.3: Hardware specifications and results on the Oxford Radar RobotCar model [2]

Specification	Intel i9-12900H	NVIDIA RTX 3060	Intel Loihi 2
Technology node	10 nm	8 nm	7 nm (Intel 4)
Core count	14	3584	128
Precision	Float32	Float32	Fixed32
Frequency (MHz)	3700	1320	1000
Memory	DDR5	GDDR6	On-chip
MOP per inference	960	960	960
Throughput (GOP/s)	8.67	304.76	161.11
Power (W)	28	80	1.55
Power efficiency (GOP/s/W)	0.30	3.80	103.94



(a) Throughput **(b)** Power efficiency

Figure 3.2.2: Throughput and power efficiency of the CPU, GPU, and Loihi 2 on the sensor-fusion model: the GPU is fastest, but Loihi 2 is far more efficient (data from [2])

Per-dataset performance. Table 3.4 reports Loihi-2's metrics on all five datasets. The networks use 1100–1200 neurons and 100K–120K synapses, draw 1.5–2.5 W, and complete inference in 0.58–0.80 ms with throughputs of 141–161 GOP/s [2]. The Oxford Radar RobotCar model is the fastest and most frugal, and the AIODrive model, with the most complex input, is the slowest and most power-hungry. The reported efficiency in pJ per operation does not follow directly from power divided by throughput—for example, 1.50 W at 161 GOP/s gives about 9.3 pJ/op rather than 11.52—so these values are best read as indicative.

Comparison with earlier implementations. The authors compare Loihi-2's inference times with published implementations on FPGAs and GPUs. On nuScenes, earlier systems need 17–700 ms on hardware ranging from an FPGA to RTX 3090 and RTX A6000 GPUs, against 0.70 ms on Loihi-2; on Oxford Radar RobotCar, Burnett et al.

Table 3.4: Performance of Loihi-2 on the five sensor-fusion datasets [2]

Dataset	Power (W)	Time (ms)	GOP/s	pJ/OP	Neurons	Synapses
AIODrive	2.50	0.80	141	10.12	1200	120K
Oxford RobotCar	1.50	0.58	161	11.52	1100	100K
D-Behavior (D-Set)	1.78	0.61	159	11.38	1100	100K
nuScenes	2.38	0.70	143	10.38	1200	120K
Comma2k19	1.98	0.65	149	11.38	1200	120K

need 70 ms on a V100 GPU and Xeon CPU against 0.58 ms; and on D-Set, Liu et al. need 330 ms against 0.61 ms [2]. On Comma2k19, however, Chen et al. report 0.01 ms on a GTX 1080, faster than Loihi-2's 0.65 ms. The earlier works also solve different, often larger, perception tasks with different models, so these ratios indicate potential rather than like-for-like speed-ups.

3.2.6 Discussion

The paper is one of the first to apply Loihi-2 to multimodal sensor fusion across several real driving datasets, and it reports sub-millisecond inference at 1.5–2.5 W with power efficiency far above a CPU and GPU. It also shares useful practical lessons on the HDF5/NetX export path and data-format mismatches. Its limitations are significant, however. The SNN architectures, training procedures, and accuracy on each dataset are not reported, so efficiency cannot be weighed against task quality; the experiments use data generated for testing in an ideal environment rather than live sensor streams; several reported figures are internally inconsistent, such as 128 cores and a 7 nm node for an Intel 4 chip that [1, 4] describe as having 120 cores; and comparisons with earlier implementations mix different tasks. The authors plan real-time sensor fusion experiments and collaboration with industry as future work [2].

3.3 A Spike-Based Long Short-Term Memory on a Neurosynaptic Processor

Shrestha, Ahmed, Wang, Widemann, Moody, Van Essen, and Qiu [3], from Syracuse University and Lawrence Livermore National Laboratory, present a design flow that maps an LSTM network onto IBM's TrueNorth neurosynaptic processor and evaluate its accuracy, energy, and speed.

3.3.1 Problem Statement and Motivation

Feedforward networks cannot retain temporal dependencies, and vanilla recurrent networks struggle to learn long dependencies because of vanishing and exploding gradients. The **LSTM** solves this with gates that control what is forgotten, added to, and output from a cell state, and is a standard model for time-series processing. TrueNorth-compatible learning algorithms had achieved close to state-of-the-art results for feedforward networks such as convolutional networks, but recurrent networks in the spike domain remained at the proof-of-concept level, because of the hardware's limited connectivity and weight precision and the difficulty of capturing recurrent dynamics with asynchronous spiking neurons [3]. To the authors' knowledge, theirs is the first implementation of an LSTM in the spike domain.

3.3.2 LSTM and the TrueNorth Architecture

A standard LSTM computes, at each step t,

$$\begin{aligned} f_t &= \sigma\big(W_f[h_{t-1}, x_t] + b_f\big), & i_t &= \sigma\big(W_i[h_{t-1}, x_t] + b_i\big), \\ C'_t &= \tanh\big(W_C[h_{t-1}, x_t] + b_C\big), & C_t &= f_t * C_{t-1} + i_t * C'_t, \\ o_t &= \sigma\big(W_o[h_{t-1}, x_t] + b_o\big), & h_t &= o_t * \tanh(C_t), \end{aligned} \tag{3.1}$$

where f_t, i_t, and o_t are the forget, input, and output gates, C_t is the cell state, and h_t is the output [3]. Implementing these equations requires matrix–vector products, two non-linearities, element-wise products, and a sum, all synchronised so that h_{t-1} meets x_t at the right moment. On TrueNorth (Figure 3.3.1), each core is a crossbar of 256 axons and 256 neurons; a group of cores with defined inputs and outputs forms a **corelet**, and corelets are composed into larger networks.

3.3.3 Constrain-Then-Train-Then-Approximate

TrueNorth weights are low-precision integers with at most four values per neuron, and the authors found that LSTMs are hard to train with binary or ternary weights. They therefore train a regular LSTM under constraints that anticipate the hardware and then approximate it, a process they call **constrain-then-train-then-approximate**. Weights are constrained to $[-1, 1]$ or $[-2, 2]$ during training so that they can later be scaled into

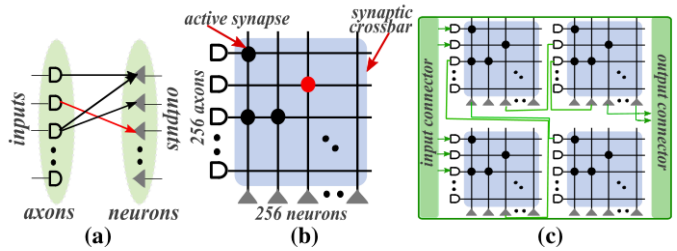


Figure 3.3.1: (a) Structural view, (b) crossbar view, and (c) corelet abstraction of a TrueNorth core [3]

the hardware range, and the gates are replaced by piecewise-linear versions with steep slopes,

$$\sigma(x) = \max\big(0, \min(1,\ x + 0.5)\big), \qquad \tanh(x) = \max\big(-1, \min(1,\ 2x)\big), \tag{3.2}$$

which are used during both training and recall. After training, the floating-point weights are scaled and rounded to the hardware precision [3]. The steep slopes shrink the linear range of the gates and limit the propagation of rounding errors.

3.3.4 Spike Encoding

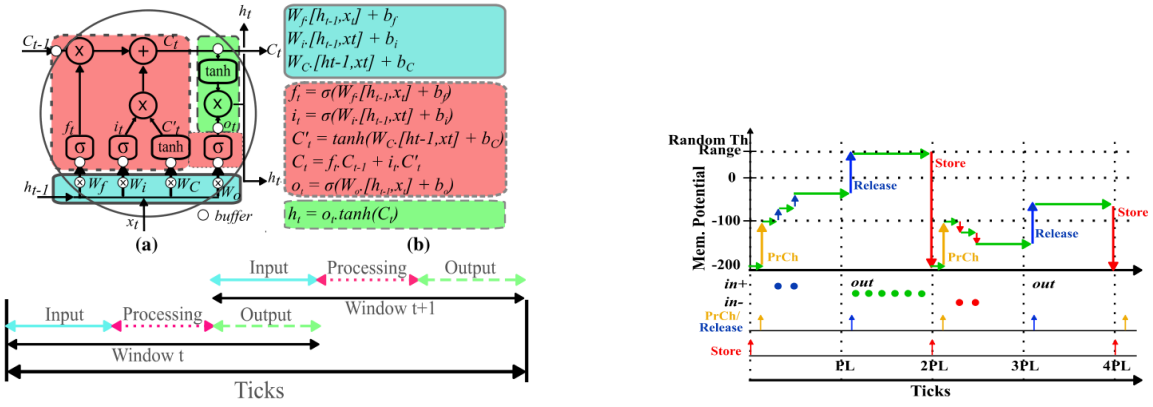
LSTM values can be negative, but spikes are binary. Replacing tanh with ReLU would make all values positive, but unbounded activations can make an LSTM diverge, so the authors instead use separate **positive and negative spike channels**. Inputs and outputs are **rate-coded** over a window called the **phase length** (PL). If mx is the maximum value to represent and the weights are scaled up by a factor sf, the number of spikes that represents the value 1 is

$$nS = \frac{PL}{mx \cdot sf}, \tag{3.3}$$

a value I_v is sent at a firing rate of $I_v \cdot nS/PL$, and n spikes received in a phase represent $RPValue = n/nS$. Each spike is therefore worth $1/nS$, so precision improves with the phase length. All internal variables are rate-coded except the cell state C_t: because errors in C_t accumulate through its feedback loop, it is represented with a **burst code**, whose spike count correlates much more closely with the value than a stochastic rate code [3].

3.3.5 Store-and-Release Synchronisation

Asynchronous neurons fire as soon as they can, but an LSTM needs its gates to operate on synchronised inputs. The authors solve this with a **store-and-release** mechanism. A store-and-release neuron has a large negative, saturating threshold and two clock inputs. A spike on the *store* clock, with a large negative weight, pushes the membrane potential to the negative threshold; the neuron then accumulates input spikes without firing. A spike on the *release* clock, with a large positive weight, raises the potential to zero or above, and the neuron emits spikes in proportion to what it stored. Because a saturated neuron cannot count negative inputs, a *pre-charge* spike on the release line first lifts the potential to an intermediate level; Figure 3.3.2b shows a neuron entering store mode through -250 and release through two pulses of $+100$ [3]. During release, the accumulated membrane potential (AMP) equals the net number of stored input spikes; at each tick a random number in $[0, R_{ThR}]$ is drawn and a spike is emitted if it is below the AMP, so about $PL \cdot AMP/R_{ThR}$ spikes are produced, and with $R_{ThR} = PL$ the neuron simply relays its input.



(a) Partitions, phases, and partial pipelining **(b)** Store-and-release neuron

Figure 3.3.2: Operation of the spike-based LSTM: input (blue), processing (red), and output (green) partitions alternate between phases, synchronised by store-and-release neurons driven by store, pre-charge, and release clocks [3]

With this mechanism the S-LSTM works in two alternating phases (Figure 3.3.2a). In the **I/O phase**, the input partition computes the matrix–vector products for the three gates and the candidate state, and the results are buffered by store-and-release neurons. In

the **processing phase**, these neurons release their contents and the processing partition computes the new cell state C_t and the partial output, which are buffered in turn. In the next I/O phase they are released to compute h_t, which is fed back to the input partition together with the next input x_{t+1}. The phases are driven by locally generated but globally consistent clock spikes, and the partitions are partially pipelined [3].

3.3.6 Constituent Modules and Mapping

Each part of the LSTM becomes a parameterised corelet. The **Input Collection (IC) module** performs the matrix–vector products: every input uses two channels, and each weight is formed by four axons with weights 1, 2, 4, and 8, approximating values from -15 to $+15$—5-bit rather than 4-bit precision thanks to the two channels—with the product routed to the positive or negative output according to its sign. The **sigmoid** and **tanh modules** are store-and-release neurons whose release rates implement the hard functions above: the sigmoid adds an offset of $nS/2$ to the accumulated potential and uses $R_{ThR} = nS$, and the tanh uses two output channels with $R_{ThR} = nS/2$. The **dot-product module** multiplies the burst-coded cell state by a stochastically rate-coded sigmoid output through a logical AND of the two spike trains, realised by an integrate-and-fire neuron with threshold 2: in a 10-tick window, 5 sigmoid spikes (0.5) and 4 input spikes (0.4) yield 2 output spikes (0.2) [3].

Because a core has only 256 axons and 256 neurons, the modules are placed by an **incremental mapping algorithm** that adds a new core only when the current one is full. IC modules grow along both axons and neurons, since their size depends on the numbers of inputs and hidden units, whereas the other modules grow only along axons. Cores are therefore added in steps, and the average number of neurons used per core rises with network size [3].

3.3.7 Experiments and Results

All networks have one hidden LSTM layer, are trained with weights constrained to $[-2, 2]$, and use $mx = 5$; the Keras/TensorFlow counterparts reach 100% accuracy in every case [3]. Two benchmarks are used. In the **parity check** problem, the network reads a binary sequence and reports after each bit whether the number of ones so far is

even or odd; LSTMs with 2, 4, and 10 units are trained on 9000 sequences of up to 20 bits and tested on 1000 sequences on TrueNorth. In the **Embedded Reber Grammar** (ERG) task, the network reads symbols generated by walking a grammar graph and predicts the possible next symbols, coded as 7-dimensional binary vectors; LSTMs with 10, 30, and 50 units are trained on 5000 sequences of up to 36 symbols and tested on 500.



(a) Parity check (2, 4, and 10 units) **(b)** Embedded Reber grammar

Figure 3.3.3: Accuracy of the spike-based LSTM on TrueNorth versus phase length; RCt denotes a rate-coded cell state and NoC a network trained without weight constraints [3]

Accuracy. Figure 3.3.3 shows that accuracy rises with phase length, which increases the precision of activations, and with the number of LSTM units. On parity, the 10-unit network is nearly perfect from a phase length of 10 ticks and reaches 100% at 20, whereas the 2-unit network needs 40 ticks. On ERG, accuracy improves sharply once precision exceeds 1/10, and at a phase length of 200 ticks the 50-unit network reaches 96.3%. Two ablations explain the design choices. A rate-coded cell state (S-LSTM-50RCt) reaches only 58%, because rounding and sampling errors accumulate around the feedback loop. A network trained without constraints (S-LSTM-50NoC) is more accurate at short phase lengths, since its scaling factor $sf = 1$ gives a higher nS, but it plateaus at 83%, below the constrained network, once long windows provide enough precision [3].

Energy. Active power on TrueNorth is the measured total minus the leakage power, and leakage is scaled by the fraction of the chip's 4096 cores that the network uses, so packing neurons densely onto few cores matters. The 10-, 30-, and 50-unit networks use 17, 18, and 28 cores, with neuron density per core increasing with size. Table 3.5 compares the 50-unit network with GPU implementations. At its normal 1 kHz tick and 0.8 V, TrueNorth needs only **56** μ**J** per sample at a phase length of 200, which is 84

times more energy-efficient than an NVIDIA Tegra X1 and 416 times more than a Titan X. Running 3.5 times faster than real time at the same voltage improves this to 165 and 817 times [3].

Table 3.5: Power and performance of a 50-unit LSTM on different platforms [3]

Device	Time/sample (ms)	Power (W)	Energy (mJ)
NVIDIA Tegra X1	1.928	2.45	4.72
NVIDIA Titan X	0.5	46.5	23.3
TrueNorth (PL = 200)	400	0.00014	0.056
TrueNorth, 3.5× real time	114	0.00025	0.0285

Latency trade-off. TrueNorth is, however, much slower: 400 ms per sample against 1.9 ms on the Tegra X1 and 0.5 ms on the Titan X. The authors attribute this mainly to the inability to train LSTMs reliably with very low precision, which forces a long phase length to reach good accuracy; a network trained for very limited precision could use $sf = 1$ and much shorter phases, bringing latency close to that of GPUs at the same or lower energy [3].

3.3.8 Discussion

The paper's strengths are its modular, reusable design—corelets for each LSTM component that can be applied to other spiking platforms—and its careful handling of synchronisation, signed values, and precision under severe hardware constraints, yielding energy two to three orders of magnitude below GPUs. Its limitations are the small benchmarks, latency hundreds of times longer than GPUs, accuracy below the floating-point model at practical phase lengths, and a single hidden layer. It nevertheless anticipates two ideas found in later work: exploiting temporal precision through spike timing [1] and carrying exact values in spikes rather than rates [4].

3.4 Sigma-Delta Neural Network Conversion on Loihi 2

Brehove, Tumpa, Kyubwa, Menon, and Narayanan [4], from ChromoLogic LLC and Pennsylvania State University, convert conventionally trained and quantised neural networks into sigma-delta neural networks that run on Loihi 2, benchmark them against NVIDIA Jetson boards, and deploy one in a field exercise.

3.4.1 Problem Statement and Motivation

Training SNNs directly to accuracies comparable to deep ANNs is difficult because the loss must be backpropagated through time; in the authors' experiments, training a spiking network took about 10 times as long as training an ANN of the same architecture. Classic ANN-to-SNN conversion uses spike rates to represent activations and therefore needs many timesteps per input, which erodes efficiency [4]. Intel's earlier sigma-delta networks on Loihi 2 relied on SLAYER, which stores the whole time series during training and is slower than conventional training, and an earlier conversion method by Yousefzadeh et al. used binary spikes and was incompatible with modern quantisation. The goal is therefore a conversion path that starts from standard PyTorch training and quantisation and produces sparse networks that exploit Loihi 2's graded spikes.

3.4.2 Sigma-Delta Neurons and the Conversion Method

A **sigma-delta neuron** [8] receives spikes that represent *changes* in the previous layer's activations, multiplies them by its weights, adds them to a running sum, and sends the change in its own activation to the next layer only if that change exceeds a threshold (Figure 3.4.1). Apart from this, it behaves exactly like a conventional ANN neuron, but because it computes only when inputs change, it naturally exhibits temporal sparsity, and because only some neurons change at a time it also exhibits spatial sparsity [4].

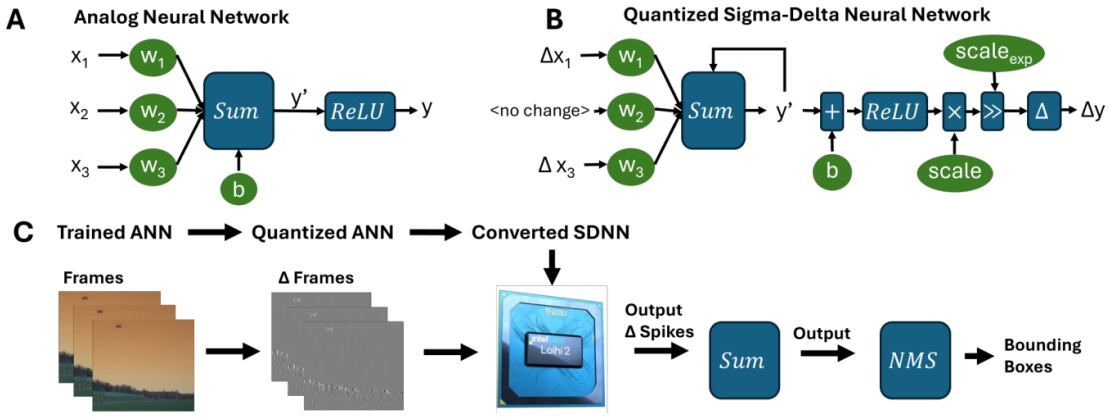


Figure 3.4.1: (A) A conventional ANN neuron; (B) a quantised sigma-delta neuron on Loihi 2, which accumulates weighted input changes, applies ReLU and integer rescaling, and emits a graded spike Δy when its output has changed by more than a threshold; (C) the workflow from trained ANN to quantised ANN to converted SDNN, with frames turned into delta spikes and output spikes summed into bounding boxes [4]

Loihi 2's neurocores compute only with integers, so the network must be quantised. The authors use PyTorch's **post-training static quantisation**, which converts weights and activations from FP32 to 8-bit integers with an affine mapping $x_{\text{float}} = s\,(x_q - z)$ defined by a floating-point scale s and an integer zero point z, while biases stay in FP32. A layer's floating-point output is then

$$y_{\text{float},j} = \sum_k \big(x_{q,k} - z_x\big)\big(w_{q,jk} - z_w\big)\, s_x\, s_w + b_{\text{float},j}, \tag{3.4}$$

and must be re-quantised with the output scale s_y for the next layer. Intel's original sigma-delta microcode could rescale only by powers of two, so the authors **redesigned the neuron microcode**: the 24-bit accumulated pre-activation is multiplied by a pre-computed 24-bit integer *scale* derived from $s_x s_w / s_y$ and right-shifted by a pre-computed exponent, $y = (acc \times scale) \gg scale_{\text{exp}}$, which keeps the result within 8 bits and prevents overflow in the next layer's 24-bit accumulator, following the integer-only inference approach of Jacob et al. [4]. Algorithm 2 gives the new neuron logic.

Algorithm 2 Redesigned sigma-delta neuron microcode on Loihi 2 [4]

Require: quantised weights w_q; rescaled bias; integer *scale* and $scale_{\text{exp}}$; threshold v_{th}

$y' \leftarrow y' + \Delta x \cdot w_q$ ▷ accumulate weighted input delta
$y \leftarrow \text{ReLU}(y' + bias)$
$y \leftarrow (y \times scale) \gg scale_{\text{exp}}$ ▷ fixed-point rescaling
$\Delta y \leftarrow y - y_{\text{ref}}$
if $|\Delta y| > v_{th}$ **then**
    emit graded spike carrying Δy; $y_{\text{ref}} \leftarrow y$
end if

At run time, an initial **delta layer** on the host CPU turns a sequence of RGB frames into graded spikes, $spike_i = frame_i - frame_{i-1}$. Loihi 2 performs the convolutions and sigma-delta neurons, and its output spikes are added to a running sum on the host, dequantised with the final layer's scale, and interpreted as a YOLOv3-style grid of 14×14 cells, each with locations and scores for three bounding boxes. Deployment uses `nxkernel` v0.3.0 in **fall-through mode**, which runs 16 SNN timesteps per input so that each frame propagates through the whole network, trading a little throughput for much lower latency than the pipelined mode [4].

3.4.3 Datasets, Models, and Training

Two applications are used. **Drone detection** uses the Multi-Sensor Drone Detection (MSDD) dataset: 285 RGB videos at 640×512, resized to 448×448, with four labelled classes—drones, airplanes, helicopters, and birds—at 20–200 m in daylight, with 20% held out for testing. Ground-based sky videos have few moving objects, so they are highly temporally sparse. **Steering-angle prediction** uses the Driving dataset, on which Intel demonstrated SLAYER-trained SDNNs; dashboard frames are resized to $66 \times 200 \times 3$ for PilotNet [4]. Table 3.6 summarises the models. YOLO-KP, developed by Intel from YOLOv3-Tiny, was adapted to Loihi 2's memory limits by replacing max-pooling with strided convolutions and removing residual layers, which also avoids synchronising spikes from different layers; PilotNet is identical to Intel's SLAYER-trained model.

Table 3.6: Models converted and deployed on Loihi 2 [4]

	PilotNet	**YOLO-KP**
Type	Conv/FC	Conv
Application	Steering-angle prediction	Object detection
Parameters	0.35M	3.4M
Neurons	12K	2M
Loihi 2 chips	1	5

To show that conversion works with any standard training recipe, YOLO-KP is trained by **knowledge distillation** from a MobileNetV2 teacher, combining the detection loss with a masked L2 feature-matching loss on the penultimate layer weighted by $\alpha = 0.069$, with Optuna hyperparameter tuning, albumentations augmentation, the Adam optimiser, and 30 epochs. The model is then quantised to 8-bit per tensor. Sparsity is controlled by the sigma-delta **threshold**: with a threshold of zero, the cumulative output exactly matches the quantised network, and raising it from 0 to 4 passes only larger changes, reducing spikes and synaptic operations at some cost in accuracy [4].

Loihi 2 is benchmarked on an in-house VPX board with 16 chips, using NxKernel's probes for power, timing, and core activity; static power is counted only for the chips the network uses, and the EDP combines energy and latency. To separate computation from

input/output (IO), 100 frames are also loaded into on-chip memory and replayed 10,000 times. The baseline runs the same network as a 16-bit floating-point ANN with TensorRT on a Jetson Xavier at batch size 1, measuring whole-SoC power with `tegrastats`; the authors note that this is not an identical comparison, since only the Loihi 2 chips' power is counted [4].

3.4.4 Results and Analysis

Accuracy and training. The MobileNetV2 teacher reaches 0.72 mAP and the distilled YOLO-KP student 0.74, whereas YOLO-KP without distillation never exceeded 0.6; after 8-bit quantisation the student scores 0.62 mAP over 58 test videos with 18,214 frames. PilotNet reaches its lowest MSE, 0.0278, at threshold 0 [4]. Against SLAYER training of the same YOLO-KP SDNN, the conversion approach is both more accurate and far faster to train: 5 minutes per epoch and 3.4 hours in total against 55 minutes and 45.8 hours, with 8.6 GB instead of 9.4 GB of memory and 0.62 instead of 0.37 mAP; conversion itself adds only 30 seconds of post-training quantisation.

YOLO-KP efficiency. Sweeping the threshold shows the sparsity trade-off. At threshold 0 the SDNN performs only 0.056 of the ANN's operations, an 18-fold sparsity gain; raising the threshold to 4 increases this to 250-fold and throughput from 176 to 362 FPS, while mAP falls. Loihi 2's power stays roughly constant because it is dominated by static draw. At threshold 0 (Table 3.7), Loihi 2 uses 11.73 mJ per frame with 3.20 ms latency, against 14.10 mJ and 3.11 ms on the Jetson Xavier, giving a modestly lower EDP but lower throughput [4].

Table 3.7: Loihi 2 compared with Jetson boards at zero threshold [4]

Model	Platform	mJ/frame	Latency (ms)	FPS	EDP (μJ·s)
YOLO-KP SDNN	Loihi 2	11.73	3.20	175.6	37.59
YOLO-KP SDNN	Loihi 2, no IO	19.89	2.86	196.5	56.93
YOLO-KP ANN	Jetson Xavier	14.10	3.11	321.9	43.80
PilotNet SDNN	Loihi 2	1.31	2.50	200.2	3.26
PilotNet SDNN	Loihi 2, no IO	1.06	1.13	442.9	1.20
PilotNet ANN	Jetson Nano	21.94	5.77	173.2	126.20

PilotNet efficiency. For PilotNet, the SDNN on Loihi 2 needs 1.31 mJ per frame and 2.50 ms, against 21.94 mJ and 5.77 ms for the ANN on a Jetson Nano, so energy is about **16 times** lower and EDP about **40 times** lower (3.26 against 126.20 μJ·s). Without IO, energy falls to 1.06 mJ, latency to 1.13 ms, and throughput rises to 442.9 FPS, giving an EDP of 1.20 μJ·s, about 105 times below the Jetson. Re-running Intel's own PilotNet SDNN with the newer NxKernel API showed similar energy to Intel's published 1.26 mJ but far lower latency than its 65.4 ms, indicating that IO and system overheads dominated the earlier pipeline. The converted PilotNet also achieves a lower MSE than the SLAYER-trained equivalent, although with somewhat less sparsity, since conversion does not explicitly encourage sparsity during training [4].

Field deployment. To test operational feasibility, YOLO-KP was trained as an ANN on the infrared subset of MSDD, converted, and deployed on the Loihi 2 VPX board at the US Navy's **Silent Swarm 2025** exercise (Figure 3.4.2). The host quantised and delta-encoded frames, sent spikes to Loihi 2 over Ethernet, and summed the output spikes for non-maximum suppression and display. The system ran in real time at **112 FPS** with **5.0 ms** latency and **20 mJ** per frame. Detection reflected a large domain gap: the MSDD model scored 0.03 mAP on the exercise data, rising tenfold to 0.30 mAP after minimal fine-tuning on a small subset. Novel drone types, more complex backgrounds, and camera motion reduced temporal sparsity, with 13% of inputs non-zero after delta encoding against 2% for MSDD [4].

3.4.5 Discussion

The work's main strength is practicality: it lets developers use the ordinary PyTorch toolchain—optimisers, augmentation, distillation, and quantisation—and still obtain sparse networks that run on Loihi 2, with better accuracy and an order of magnitude faster training than SLAYER, and it validates the result in a live field deployment. Its limitations are that gains depend strongly on temporal sparsity, being large for PilotNet but modest for YOLO-KP, whose throughput is lower than the Jetson's; that Loihi 2's on-chip SRAM raises static power, which grows with the number of chips needed for high-resolution inputs; and that the Jetson comparisons use whole-SoC power and, for

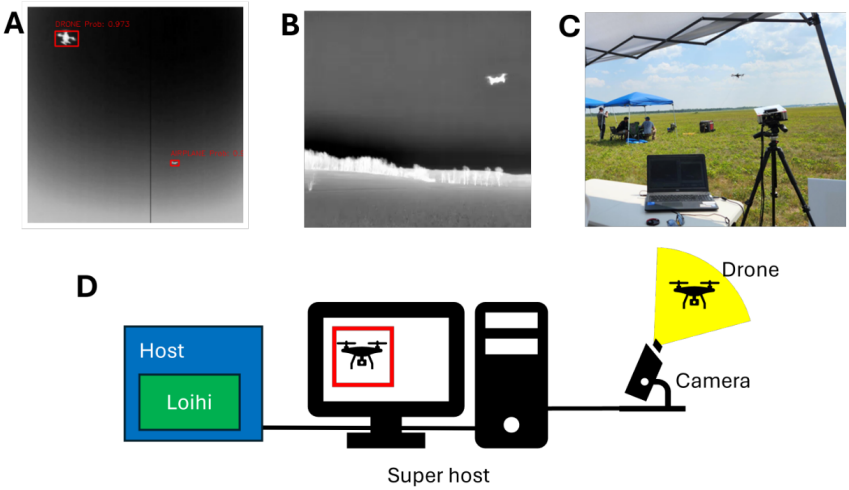


Figure 3.4.2: Field deployment at Silent Swarm 2025: (A) infrared frame with a drone and an airplane, (B) a training frame from MSDD, (C) the field setup with an IR camera and the Loihi 2 VPX board, and (D) the system diagram [4]

PilotNet, Intel's earlier measurements. The authors suggest pairing SDNNs with event-based cameras and using larger single chips to reduce costly inter-chip communication [4].

3.5 Summary of the Reviewed Papers

Table 3.8 summarises the four works; the next chapter compares them in detail.

Table 3.8: Summary of the reviewed works

Work	Problem	Technique	Headline result
Meszaros et al. [1]	Keyword spotting on Loihi 2	EventProp delay learning in mlGeNN, NetX export, NxKernel deployment	Up to 26.9-point accuracy gain from delays; up to 18× faster and ∼250× less energy than Jetson Orin Nano
Isik et al. [2]	Multimodal sensor fusion for driving	Five dataset-specific SNNs with Lava-DL on Loihi-2	103.94 GOP/s/W; 0.58–0.80 ms inference at 1.5–2.5 W
Shrestha et al. [3]	Recurrent networks on TrueNorth	Constrain-train-approximate LSTM; burst-coded cell state; store-and-release	56 μJ/sample; 84–817× more efficient than GPUs
Brehove et al. [4]	Efficient ANN-to-SNN deployment	Quantised ANN to sigma-delta SNN with redesigned microcode	PilotNet 16× lower energy, ∼40× lower EDP; field deployment at 112 FPS

Chapter 4

Discussion & Comparison

This chapter compares the four reviewed works. All deploy spiking networks on neuromorphic processors and compare them with conventional hardware, but they differ in how they obtain the network, how they encode information, which chip they target, and how they measure efficiency. The comparison highlights their complementary strengths and the trade-offs among accuracy, energy, latency, and ease of use.

4.1 Analytical Focus and Methodology

The works examine neuromorphic deployment from four angles: **learning temporal structure** through trainable delays and bringing it to the chip [1]; the **system-level benefit** of neuromorphic hardware for a demanding multimodal application [2]; **building a complex recurrent architecture** from spiking circuits under severe hardware constraints [3]; and **reusing the conventional deep-learning toolchain** through conversion [4]. Together they cover the path from training algorithm to field deployment. Table 4.1 compares their methodologies.

4.2 Training and Conversion Strategies

The works illustrate the main routes to a deployable SNN. **Direct training** [1] gives the most freedom: EventProp learns weights and delays jointly with exact gradients and produces networks that genuinely compute in time, but it needs a specialised toolchain—mlGeNN, NetX, and NxKernel. **Constrain-then-train-then-approximate** [3] uses conventional training but needs hand-designed circuits, and residual approximation errors must be paid for with long phase lengths. **Conversion** [4] is the most accessible: any PyTorch model that can be quantised can, in principle, run on Loihi 2, and it trained 11 times faster per epoch than SLAYER with higher mAP. Paper [2] uses the Lava-DL

Table 4.1: Methodological comparison of the reviewed works

Aspect	**Meszaros et al. [1]**	**Isik et al. [2]**	**Shrestha et al. [3]**	**Brehove et al. [4]**
Hardware	Loihi 2 (1 chip)	Loihi-2	TrueNorth	Loihi 2 (1–5 chips)
Network	FF and recurrent LIF SNN	Five dataset-specific SNNs	One-layer LSTM	YOLO-KP, PilotNet
How obtained	Direct EventProp training	Lava-DL training, NetX export	Constrain, train, approximate	Quantised ANN conversion
Coding	Binary spikes, delays	Binary spikes	Rate and burst codes	Graded sigma-delta spikes
Task	Keyword spotting	Sensor fusion	Parity, Reber grammar	Detection, steering
Baseline	Jetson Orin Nano	i9 CPU, RTX 3060 GPU	Tegra X1, Titan X	Jetson Xavier, Nano
Accuracy reported	Yes	No	Yes	Yes

route and documents its practical friction. The more a method reuses standard tools, the easier it is to adopt, but the less it exploits uniquely spiking features such as delays.

4.3 Neural Coding and Sparsity

Coding determines how many spikes—and therefore how much energy—a computation needs. Rate coding [3] needs many spikes per value, which is why the S-LSTM needs 200-tick phases and 400 ms per sample, although burst coding improves fidelity for the cell state. Binary spikes with learned delays [1] encode information in timing at low firing rates, and graded sigma-delta spikes [4] carry exact integer changes and fire only when inputs change, giving 18-fold fewer operations than the ANN without loss of fidelity. The trend is from many binary spikes per value towards few spikes, each carrying more information.

4.4 Hardware Platforms and Mapping Constraints

Each chip imposes constraints that shape the algorithm. TrueNorth's four weight values per neuron, 256-input cores, and fixed neuron model forced [3] to build gates from multiple axons and special circuits and to pack neurons densely to limit leakage. Loihi 2 is more flexible, but its 62-step delay limit costs [1] about 5% accuracy, its integer-only cores

required [4] to rewrite the neuron microcode for fixed-point rescaling, and its per-core neuron capacity led [4] to remove max-pooling and residual layers and spread YOLO-KP over five chips. The evolution from TrueNorth to Loihi 2—programmable neurons, graded spikes, and native delays—has steadily reduced how much an algorithm must be distorted to fit the hardware.

4.5 Energy, Latency, and Accuracy Trade-offs

All four works report large efficiency gains (Figure 4.5.1), but against different baselines and metrics. Loihi 2 uses about 200 to 260 times less energy per keyword than a Jetson Orin Nano at batch size 1 [1]; Loihi-2 is about 27 times more power-efficient than an RTX 3060 GPU and over 300 times more than an i9 CPU on sensor fusion [2]; TrueNorth is 84 to 817 times more energy-efficient than Tegra X1 and Titan X GPUs [3]; and the PilotNet SDNN uses 16 times less energy than a Jetson Nano, with about 40 times lower EDP [4]. These gains are real, but they come with trade-offs. **Latency** varies widely: Loihi 2 is 18 times *faster* than the Jetson in [1], whereas TrueNorth is hundreds of times *slower* than GPUs in [3]. **Throughput** can favour the GPU, as for YOLO-KP in [4] and on raw GOP/s in [2]. **Accuracy** is preserved after 8-bit quantisation in [1], decreases modestly in [4], depends on phase length in [3], and is not reported in [2]. The gains also depend on **sparsity**: they are largest for small, sparse workloads such as keyword spotting and steering, and much smaller for high-resolution detection that needs several chips.

4.6 Benchmarking Practices and Reproducibility

The works measure efficiency in different ways: [1] uses an IO-unconstrained mode on Loihi 2 and `jtop` power over several Jetson rails; [4] reports IO-bound and IO-free results and whole-SoC Jetson power; [3] scales TrueNorth's leakage to the cores used; and [2] does not describe its measurement method. Paper [4] shows how much this matters: Intel's PilotNet SDNN had similar energy but 65.4 ms latency in earlier measurements, against 2.50 ms with the newer software stack, because IO and system overheads had dominated. Fair evaluation therefore requires reporting accuracy alongside efficiency, separating IO from computation, stating which power rails are counted, and using well-tuned baselines at realistic batch sizes.

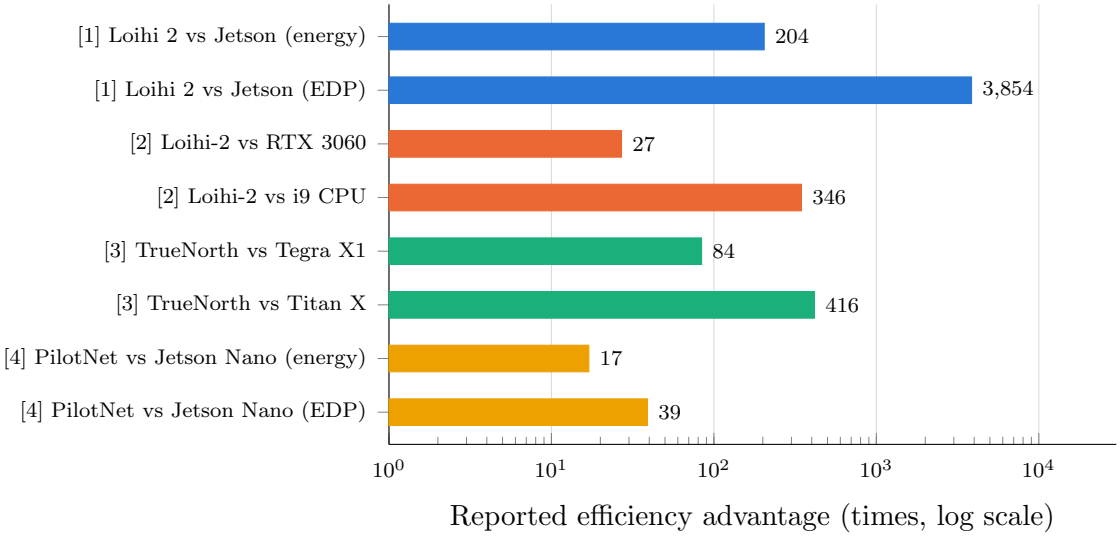


Figure 4.5.1: Efficiency advantages over conventional hardware reported in the reviewed works, computed from their published tables: energy or EDP per inference for [1, 3, 4] (feedforward delay network at batch size 1 for [1]) and power efficiency in GOP/s/W for [2]

4.7 Overall Comparative Insights

A consistent pattern emerges. **First**, neuromorphic hardware delivers order-of-magnitude gains in energy and EDP when network activity is sparse and computation is mapped to the chip's native primitives—delays [1], crossbar cores [3], or graded spikes [4]. **Second**, these gains are traded against accuracy, latency, or throughput in ways that depend on the coding scheme and on IO overheads. **Third**, ease of use is improving: conversion from standard PyTorch models [4] and export through NetX [1, 2] lower the barrier that TrueNorth's hand-built circuits [3] once represented. **Fourth**, evaluation practices are inconsistent, and system-level studies must report task accuracy and measurement details to be convincing. Hybrid pipelines that combine conventional training, hardware-aware conversion, and native temporal features such as delays are therefore the most promising path.

Chapter 5

Applications

5.1 Keyword Spotting and Voice Interfaces

Devices that listen continuously for wake words benefit directly: delay SNNs on Loihi 2 classify spoken digits in about 1.5 ms with under half a millijoule per sample [1], suiting always-on voice control, acoustic event detection, and hearing aids.

5.2 Autonomous Driving and Sensor Fusion

Vehicles fuse cameras, LIDAR, radar, GPS, and IMUs under tight power and latency budgets. Sub-millisecond sensor fusion at 1.5–2.5 W [2] and 40-fold lower EDP for PilotNet steering [4] suit always-on perception.

5.3 Time-Series and Sequence Processing

Spike-based LSTMs at micro-joule energy [3] and delay networks [1] provide memory for physiological, industrial, and network streams, enabling anomaly detection and predictive maintenance.

5.4 Drone Detection and Surveillance

Sky-watching sensors produce highly sparse video because only the targets move; a converted SDNN on Loihi 2 ran in real time at 112 FPS and 20 mJ per frame at a US Navy exercise [4].

5.5 Robotics and Edge IoT

Robots, drones, and IoT nodes paired with event cameras and tactile sensors can feed sparse data directly to neuromorphic units placed between sensors and actuators [2], enabling low-power obstacle avoidance and gesture recognition.

Chapter 6

Challenges and Future Directions

Several obstacles remain before neuromorphic computing becomes a mainstream option for edge AI.

6.1 Challenges Identified in the Reviewed Works

Meszaros et al. [1] are limited by the 62-step delay maximum, which costs about 5% accuracy, and by single-seed, IO-unconstrained Loihi results. **Isik et al.** [2] report no task accuracy and use test data from an ideal environment. **Shrestha et al.** [3] need long phase lengths and hundreds of milliseconds per sample. **Brehove et al.** [4] gain little for dense, high-resolution video and face static power that grows with the number of chips.

6.2 Cross-Cutting Challenges

The field faces **training complexity**, since direct SNN training is less mature than deep learning; **precision** losses from low-precision weights and spike coding; **IO overheads** that can dominate latency and energy [4]; **scalability** limits from inter-chip communication and static power; fragmented **tooling** and inconsistent **benchmarking**; and a **sensor mismatch**, as frame-based cameras do not produce sparse events.

6.3 Future Research Directions

Promising directions are **event-based sensing** to supply naturally sparse inputs [4]; **richer temporal computation** with longer delays and adaptive neurons [1]; **hybrid training** that converts a conventional network and fine-tunes it with spike-aware methods; **real-time evaluation** on live sensor streams with reported accuracy [2]; **low-precision recurrent models** with short phase lengths [3]; and **standard benchmarks** and **on-chip learning** for fair comparison and adaptation in the field.

Chapter 7

Conclusion

Conventional deep-learning hardware spends most of its energy moving data and recomputing unchanged inputs, which suits always-on edge applications poorly. Neuromorphic computing addresses this by computing with sparse spikes on chips that co-locate memory and processing. This seminar reviewed four works that train, convert, and deploy spiking neural networks on IBM's TrueNorth and Intel's Loihi 2.

Paper [1], the base paper, built a pipeline from EventProp delay learning to Loihi 2, raising feedforward keyword-spotting accuracy by up to 26.9 points and running up to 18 times faster with about 250 times less energy than a Jetson Orin Nano. **Paper [2]** applied Loihi-2 to sensor fusion on five driving datasets at 103.94 GOP/s/W with sub-millisecond inference. **Paper [3]** mapped an LSTM onto TrueNorth at 56 μJ per sample. **Paper [4]** converted quantised PyTorch networks into sigma-delta networks, achieving 16 times lower energy and about 40 times lower EDP for PilotNet and a real-time field deployment. The comparison shows that **sparsity is the source of efficiency**, that **coding matters**, since few, information-rich spikes reduce latency and energy together, that **toolchains decide adoption**, and that **evaluation must be complete**, reporting accuracy and IO costs alongside efficiency.

Across training strategy, coding, hardware constraints, and measured efficiency, a consistent pattern emerges: neuromorphic hardware delivers order-of-magnitude gains in energy and energy-delay product when network activity is sparse and computation uses the chip's native primitives, while accuracy, latency, and input-output overheads remain the main trade-offs. Pairing conventional deep-learning toolchains with hardware-aware conversion and event-based sensors is the most practical path to efficient deployment of spiking neural networks on Loihi 2 and TrueNorth.

References

[1] B. Meszaros, J. C. Knight, J. Timcheck, and T. Nowotny, "A complete pipeline for deploying SNNs with synaptic delays on Loihi 2," arXiv preprint arXiv:2510.13757, Oct. 2025.

[2] M. Isik, K. Tiwari, M. B. Eryilmaz, and I. C. Dikmen, "Accelerating sensor fusion in neuromorphic computing: A case study on Loihi-2," arXiv preprint arXiv:2408.16096, Aug. 2024.

[3] A. Shrestha, K. Ahmed, Y. Wang, D. P. Widemann, A. T. Moody, B. C. Van Essen, and Q. Qiu, "A spike-based long short-term memory on a neurosynaptic processor," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2017, pp. 631–637.

[4] M. Brehove, S. A. Tumpa, E. Kyubwa, N. Menon, and V. Narayanan, "Sigma-delta neural network conversion on Loihi 2," in *Proc. Int. Conf. Neuromorphic Systems (ICONS '26)*, Chicago, IL, USA, Aug. 2026, arXiv:2505.06417.

[5] P. A. Merolla, J. V. Arthur, R. Alvarez-Icaza, A. S. Cassidy, J. Sawada, F. Akopyan, *et al.*, "A million spiking-neuron integrated circuit with a scalable communication network and interface," *Science*, vol. 345, no. 6197, pp. 668–673, 2014.

[6] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[7] G. Orchard, E. P. Frady, D. B. D. Rubin, S. Sanborn, S. B. Shrestha, F. T. Sommer, and M. Davies, "Efficient neuromorphic signal processing with Loihi 2," in *Proc. IEEE Workshop on Signal Processing Systems (SiPS)*, 2021, pp. 254–259.

[8] P. O'Connor and M. Welling, "Sigma delta quantized networks," arXiv preprint arXiv:1611.02024, 2016.

[9] S. B. Shrestha and G. Orchard, "SLAYER: Spike layer error reassignment in time," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[10] T. C. Wunderlich and C. Pehle, "Event-based backpropagation can compute exact gradients for spiking neural networks," *Scientific Reports*, vol. 11, no. 1, p. 12829, 2021.